CS3500 Operating Systems

Locks Lab

Due: 11:59 PM, 22 October 2026

1 Overview

In this lab you will reduce lock contention in the xv6 kernel. A lock that many CPUs acquire at the same time forces them to wait for one another, so a multi-core machine behaves like a single-core one. The usual cure is to redesign the data structure so that different CPUs need different locks, or to use a lock that lets compatible operations proceed in parallel.

The lab has two independent parts:

1. Redesign the physical-page allocator so that each CPU has its own free list and its own lock.
2. Implement a *reader–writer spinlock* that admits many readers at once, admits at most one writer, and gives waiting writers priority.

2 Background Reading and Environment Setup

Before starting, read the *Locking* chapter of the xv6 book and Section 3.5, "Code: Physical memory allocator". Review the following files:

- `kernel/kalloc.c` and `kernel/param.h`;
- `kernel/spinlock.c` and `kernel/spinlock.h`;
- `kernel/proc.c`, especially `cpuid()`, `push_off()` and `pop_off()`; and
- `kernel/sprintf.c`.

The starter code is the unmodified `lock` branch of the MIT xv6 labs repository. It is available in the lab repository:

`https://github.com/mishra-suyash/CS3500-Locks-Lab`

Clone it, then use the same xv6 Docker environment as in the previous labs, with the cloned directory mounted:

```
$ git clone https://github.com/mishra-suyash/CS3500-Locks-Lab
$ docker run -it -v <path-to-cloned-repo>:/home/xv6-labs \
    nandhagk/xv6-tools:latest
```

Compiler warning in the starter code

The compiler in the Docker image rejects an unused variable in `user/usertests.c` with `error: variable 'fsblocks' set but not used`. If your build stops with this error, append the following line to the end of the `Makefile` and commit it with your solution:

```
CFLAGS += -Wno-error=unused-but-set-variable
```

Run `make qemu` once before making changes. The unmodified starter should boot, and `usertests -q` should pass. `kalloctest` is expected to fail, and `rwlktest` is expected to stop with `panic: acquire`, because the starter reader–writer lock is an ordinary spinlock that the same CPU tries to acquire twice.

3 Memory Allocator

The program `user/kalloctest` stresses the physical-page allocator: several processes on different CPUs repeatedly grow and shrink their address spaces with `sbrk()` and fork short-lived children. Every allocation and free ends in `kalloc()` or `kfree()`.

In the starter code, `kalloc()` and `kfree()` share a single free list protected by a single lock, `kmem.lock`. Every allocation on every CPU therefore serializes on that one lock. `kalloctest` reports this contention by printing, for each lock, how many times `acquire()` was called and how many times it had to spin (`#test-and-set`). A run of the starter produces output such as:

```
$ kalloctest
start test1
test1 results:
--- lock kmem stats
lock: kmem: #test-and-set 17695 #acquire() 433061
--- top 5 contended locks:
...
tot= 17695
test1 FAIL
```

3.1 Per-CPU free lists

Change the allocator so that:

1. each CPU has its own free list, protected by its own lock;
2. `kfree()` returns a page to the free list of the CPU that is running;
3. `kalloc()` takes a page from the running CPU's free list;
4. when the running CPU's list is empty, `kalloc()` *steals* free pages from another CPU's list; and
5. `kalloc()` may return 0 only if, at some moment during the call, no CPU's list held a free page. This must hold even while other CPUs are allocating, freeing and stealing at the same time.

No page may be lost, and no page may be handed to two callers.

Allocations and frees on different CPUs must then run in parallel. All of your allocator locks must be initialized with `initlock()` and their names must start with `kmem`; `kalloctest` only counts locks whose names begin with that prefix.

3.2 What the tests check

`kalloctest` runs four tests. test1 and test4 measure contention: they pass when the total number of `kmem` spins stays below a threshold. test2 and test3 check that no page is lost.

- **test1**: two processes allocate and free single pages concurrently; almost no spinning is allowed.
- **test2**: repeatedly allocates every free page and then frees them all; the number of free pages must not change between rounds.
- **test3**: one process forks many short-lived children while another repeatedly allocates and frees all free memory; afterwards no page may be missing.
- **test4**: three CPUs allocate single pages while a process on a fourth CPU repeatedly allocates and frees *nearly all* of physical memory. Free lists become empty often, so stealing is frequent.

After your change, the output should look like:

```
$ kalloctest
start test1
test1 results:
--- lock kmem stats
lock: kmem0: #test-and-set 0 #acquire() 39576
lock: kmem1: #test-and-set 0 #acquire() 132391
lock: kmem2: #test-and-set 0 #acquire() 136919
lock: kmem3: #test-and-set 0 #acquire() 124198
...
tot= 0
test1 OK
start test2
total free number of pages: 32466 (out of 32768)
..........
test2 OK
start test3
..........child done 10000

test3 OK
start test4
...
test4 OK
```

Your exact counts will differ. The `#acquire()` counts include everything since boot, and the test1 processes are not pinned to CPUs, so every CPU's `kmem` lock shows activity. Locks other than the `kmem` locks may still show contention; it does not count towards the result. Your allocator must also pass `usertests sbrkmuch` and `usertests -q`.

3.3 Hints

- Use the constant `NCPU` from `kernel/param.h`.
- `kinit()` runs on one CPU. Let `freerange()` give all free memory to that CPU; the other CPUs obtain pages by stealing.
- `cpuid()` returns the current CPU number, but the result is reliable only while interrupts are disabled. Surround the code that uses it with `push_off()` and `pop_off()`.
- `snprintf()` in `kernel/sprintf.c` can build the lock names. A lock keeps a pointer to its name, so the name must not live on the stack.
- Do not hold two `kmem` locks at once if another CPU might acquire the same two locks in the opposite order.

- Keep critical sections short, especially while holding *another* CPU's lock during a steal. Anything that takes time proportional to the length of a free list will make that CPU's own `kalloc()` and `kfree()` spin. **test4** is designed to expose this.
- Pages can move from one CPU's list to another while you are looking for a CPU to steal from. A scan that finds every list empty therefore does not, by itself, prove that memory is exhausted.
- `make clean; make KCSAN=1 qemu` builds the kernel with a race detector that can help locate unprotected accesses. Run `make clean` again afterwards, before building normally.

4 Reader–Writer Lock

Some data is read far more often than it is written. The global `ticks` counter, for example, is written once per timer interrupt but read by `sys_pause()` and `sys_uptime()`. With an ordinary spinlock, two readers exclude each other although they could safely run at the same time.

A reader–writer lock has two modes. Implement the following interface in `kernel/spinlock.c`:

```
void initrwlock(struct rwspinlock *rwlk);
void read_acquire(struct rwspinlock *rwlk);
void read_release(struct rwspinlock *rwlk);
void write_acquire(struct rwspinlock *rwlk);
void write_release(struct rwspinlock *rwlk);
```

The public functions already disable interrupts and call four static functions: `read_acquire_inner()`, `read_release_inner()`, `write_acquire_inner()` and `write_release_inner()`. Replace these and `initrwlock()` with your implementation, and redefine `struct rwspinlock` in `kernel/spinlock.h` as you need. The starter versions simply wrap an ordinary spinlock.

4.1 Required semantics

Your lock must satisfy all of the following:

1. any number of readers may hold the lock at the same time;
2. at most one writer may hold the lock, and no reader may hold it while a writer does;
3. **writer priority**: once a writer has started `write_acquire()`, a reader that calls `read_acquire()` afterwards must wait until that writer has acquired and released the lock. This must hold while *any* writer is waiting, including when two writers wait at the same time, so a stream of readers can never starve a writer;
4. a CPU may hold several reader–writer locks at once, including a mixture of read and write holds, and may release them in any order;
5. operations on one lock never wait for another lock: keep all of a lock's state in its `struct rwspinlock`; and
6. `initrwlock()` sets up the lock completely. It may be called on memory with arbitrary contents, such as a lock on the stack or in a `kalloc()`ed page.

A CPU may acquire a lock for reading again while it already holds it for reading, as long as no writer is waiting; `rwlktest` does this. You do not need to support acquiring a lock you already

hold for writing, or acquiring a lock you already hold for reading while a writer is waiting: both deadlock under writer priority. Waiting writers may be served in any order. You may build the lock from atomic operations or around an embedded `struct spinlock`. Table 1 summarizes when a request may proceed.

Current state of the lock	`read_acquire()`	`write_acquire()`
Free, no writer waiting	proceeds	proceeds
Held by readers	proceeds only if no writer is waiting	waits
Held by a writer	waits	waits
A writer is waiting	waits	waits; waiting writers go in any order

Table 1: When reader and writer requests may proceed.

Check and act together

A reader that first checks "no writer is waiting" and then, in a separate step, records itself as a reader can be overtaken by a writer between the two steps. The decision to enter and the change of the lock's state must form one atomic step.

4.2 Testing

`rwlktest` runs a staged test in the kernel (`sys_rwlktest()` in `kernel/spinlock.c`) on four CPUs. It checks concurrent readers, mutual exclusion for writers, single and multiple waiting writers, and holding several locks at once. Read that function to see what each stage expects. A correct implementation prints:

```
$ rwlktest
rwspinlock_test: step 1: initrwlock
rwspinlock_test: step 2: concurrent read_acquire
...
rwspinlock_test: step 30: done
rwspinlock_test(0): 0
rwspinlock_test(1): 0
rwspinlock_test(2): 0
rwspinlock_test(3): 0
rwlktest: 4/4 CPUs succeeded
```

The four `rwspinlock_test(n)` lines may appear in any order.

4.3 Hints

- Use the GCC `__atomic` built-ins, such as `__atomic_load_n()`, `__atomic_fetch_add()` and `__atomic_compare_exchange_n()`. Their documentation is at `https://gcc.gnu.org/onlinedocs/gcc/_005f_005fatomic-Builtins.html`.
- Use acquire ordering when taking the lock and release ordering when releasing it, so that accesses inside the critical section stay inside it.
- A single flag cannot record that two writers are waiting.

- Calling `panic()` on an unbalanced release (for example, `read_release()` when there are no readers) makes bugs much easier to find.

5 Testing and Checkpoint

By the end of the first week, your implementation should pass `kalloctest`. Before submission, run:

```
$ make grade
```

This runs `kalloctest`, `usertests sbrkmuch`, `rwlktest` and `usertests -q`, and checks for `time.txt`.

Submissions are also graded with **hidden tests** that are not part of the repository. They check the requirements in Sections 3.1 and 4.1 more thoroughly than the public tests, for example that no page is lost or handed out twice, that `kalloc()` fails only when memory is truly exhausted, and that every listed property of the reader–writer lock holds. Passing `make grade` does not guarantee passing the hidden tests. Table 2 lists the points.

Test	Points
`kalloctest`: test1–test4 (10 each)	40
`kalloctest`: `usertests sbrkmuch`	10
`rwlktest`	30
`usertests -q`	19
`time.txt`	1
`make grade` total	100
Hidden tests	30
Total	130

Table 2: Points for the lab.

`usertests -q` takes a few minutes. The contention tests depend on timing, so run `make grade` more than once before you submit.

6 Submission

6.1 Time spent

Create `time.txt` and put in it a single integer: the number of hours you spent on the lab.

6.2 References and collaboration

Create `reference.txt`. List the sources you consulted and any students with whom you collaborated, including the specific topics discussed. If you used generative AI to understand, debug, or generate any portion of your work, state where it was used and include the relevant prompt logs.

6.3 Submitting

Commit every file needed by your solution, including `time.txt` and `reference.txt`, then run:

```
$ make grade
$ make zipball
```

`make zipball` produces `lab.zip`. Untracked files are not included in it; inspect `git status` and the messages printed by `make zipball`. Rename `lab.zip` to `<ROLL_NUMBER>_OS_LAB_6_LOCKS.zip` and upload it to Moodle. The submitted archive must build in the provided environment.

7 Acknowledgement

This assignment is adapted from the MIT 6.1810 Fall 2025 Locks lab. The xv6 teaching materials and lab framework were developed by the MIT PDOS group.